

App Development Project Report

Trekking Management System

1. Student Details

Name: Aman Sachin Kujur

Roll Number: 23F3002457

Email: 23f3002457@ds.study.iitm.ac.in

About Me:

I am a BS Degree student at IIT Madras pursuing Data Science and Applications. I have a strong interest in software development, machine learning, and building scalable web applications using modern frameworks.

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

Develop a role-based web application for managing trekking activities where Admins, Trek Staff, and Users interact through a secure system supporting trek management, booking, background jobs, caching, and reporting.

Approach:

The application follows a client-server architecture with Vue.js as the frontend and Flask as the backend. JWT authentication is used for role-based access, SQLite stores application data, Celery handles asynchronous tasks, Redis is used for caching and background task brokering, and Flask-Mail sends automated emails.

3. AI/LLM Declaration

AI assistance (ChatGPT - GPT-5) was used throughout the project for Bootstrap/basic UI, understanding concepts, debugging errors, generating documentation, and reviewing implementation.

The overall AI usage is estimated to be approximately **8-15%**. All implementation decisions, integration, testing, debugging, and final verification were performed manually by me, and I understand the submitted code and architecture.

4. Technologies and Frameworks Used

Technology	Purpose
<i>Flask</i>	Backend REST API
<i>Vue.js</i>	Frontend user interface
<i>Bootstrap</i>	Responsive UI styling
<i>SQLite</i>	Database
<i>SQLAlchemy</i>	ORM for database operations
<i>Flask-JWT-Extended</i>	Authentication and authorization
<i>Flask-Mail</i>	Email notifications
<i>Celery</i>	Background task processing
<i>Redis</i>	Celery broker and caching
<i>Flask-Caching</i>	Redis-based API caching

5. Database Schema / ER Diagram

The application uses three primary database tables.

User

Stores information for all users, including Admins, Trek Staff, and Trekkers. Role-based access is managed using the role field. The table also stores authentication details and account status.

Trek

Stores trek details such as trek name, location, difficulty, duration, available slots, assigned staff member, status, start date, end date, and description.

Bookings

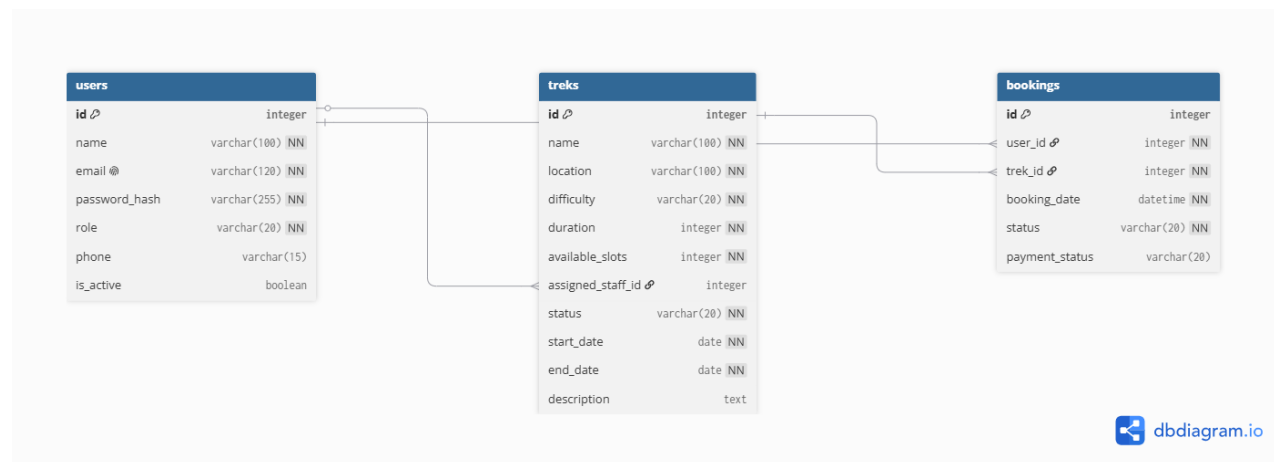
Stores booking records created by users. Each booking links a user to a trek and stores the booking date, booking status, and payment status.

Relationships

- **One User → Many Bookings**
- **One Trek → Many Bookings**
- **One User (Staff) → Many Assigned Treks**

The schema uses foreign key relationships to maintain referential integrity while reducing data redundancy.

ER Diagram:



6. API Resource Endpoints

Endpoint	Method	Description
/register	POST	Register a new user account
/login	POST	Authenticate user and generate JWT token
/profile	GET, PUT	View and update logged-in user's profile
/treks	GET, POST	View all treks or create a new trek (Admin)
/treks/<id>	GET, PUT, DELETE	Retrieve, update or delete a trek
/bookings	GET, POST	View booking history or book a trek
/bookings/export	POST	Export booking history asynchronously as CSV
/staff	GET, POST	Manage trek staff (Admin)

Endpoint	Method	Description
/staff/treks	GET	View treks assigned to logged-in staff
/admin/users	GET	View all registered users
/admin/statistics	GET	Retrieve dashboard statistics for Admin

7. Architecture and Features (optional)

Architecture Overview

- **Vue.js** – Frontend Single Page Application (SPA) for Admin, Staff, and User interfaces.
- **Vue Router** – Handles client-side navigation between application pages.
- **Flask** – Backend REST API handling authentication, trek management, bookings, and user management.
- **SQLAlchemy** – ORM used to interact with the SQLite database.
- **SQLite** – Stores application data including users, treks, and bookings.
- **Flask-JWT-Extended** – Provides JWT-based authentication and role-based authorization.
- **Celery** – Executes asynchronous background jobs such as reminder emails, monthly reports, and CSV exports.
- **Redis** – Used as the Celery message broker and backend for Flask-Caching.
- **Flask-Mail** – Sends automated emails to users and administrators.

Implemented Features

- User registration and login
- JWT-based authentication and role-based access control
- Admin management of treks, staff, users, and bookings
- Staff management of assigned treks and participant lists
- User trek booking, cancellation, profile management, and booking history
- Search and filtering of available treks
- Daily reminder emails using Celery
- Monthly HTML activity report emailed to the Admin
- Asynchronous booking history export as CSV
- Redis caching for frequently accessed trek data
- Prevention of duplicate bookings and overbooking
- Booking allowed only for treks with Open status
- Complete trekking history maintained for every user
- Email notifications using Flask-Mail
- Redis-based API caching with automatic cache invalidation

- Dashboard statistics for the Admin
- Basic Responsive user interface built using Bootstrap

Additional Features:

NONE

8. Video Presentation

DriveLink:

<https://drive.google.com/file/d/1c20egtF7QQG2Mc1XkVJMLW5BX4Xmy9Ah/view?usp=sharing>
